重邮802数据结构模拟卷（三）参考答案

对应原《模拟3.pdf》，卷内标题为“2026年硕士研究生入学考试模拟题（九）”。共23小题、150分。原卷有缺符号、重排文字及一题互相矛盾的遍历序列；相应答案会明确区分代码实际行为与可能的出题意图。

一、程序阅读题（20分）

1．带头结点单链表逆置填空（5分）

①p!=NULL；②p->next=L->next；③p=r。

先保存p原后继到r，再把p头插到结果表，最后继续处理r。题面r-p->next应改为r=p->next。原头结点L保留；时间O(n)，辅助空间O(1)。

2．队列操作输出（5分）

按明显排版笔误修正后，连续输出char。

前三次入队后为h、r、c；第一次出队使x='h'，再把h入队；第二次出队使x='r'，再入队a，得到c、h、a。循环输出c、h、a，最后输出x='r'。

原题While、缺右括号及cout<<cy等须按while、完整条件、cout<<y修正；照印刷文本不能编译。两次InitQueue都发生在入队前，不改变该结果。

3．叶结点统计及改条件后的真实行为（5分）

（1）原条件为左右孩子均空，n初始为0时统计叶结点数。若n初值非0，函数在其基础上累加叶数。

（2）改为“左空或右空”后，在每条递归分支第一次遇到缺少至少一个孩子的结点时计1，并停止向它的子树递归。因为递归调用仍放在else内。

因此不能笼统说成“统计原树中所有度为0或1的结点”。例如三个结点形成单左链，修改后的代码只在根处加1，根以下两个结点都不访问。等价理解是：把每个缺孩子结点及其后代剪成一个叶结点后，统计剪枝树的叶数。

若出题本意是统计原树所有度≤1的结点，必须把两次递归移到if之后，使计数后仍继续访问左右子树。

4．指针形参与两变量交换（5分）

（1）把main中的fun2修正为已定义的fun后，输出10 20。fun只交换两个局部指针形参，没有写入它们所指向的整数；函数返回后a、b不变。若不修正fun2，题目没有给出该函数定义，不能声称原程序可以正常链接运行。

（2）若按“交换a、b的值”的通常出题意图，只用这两个整型变量可以写：

```
a ^= b;
b ^= a;
a ^= b;
```

执行后a=20、b=10。a、b是两个不同变量，不会产生同一对象异或归零问题，也避免了加减交换可能出现的有符号溢出。若按fun的实际外部效果理解，则无需任何语句即可保持a、b原值；题目“完成fun功能”的措辞与值交换意图不一致。

二、填空题（30分）

1．循环队列长度（3分）

61。按物理数组长度70、保留一个空槽的通常约定，(rear-front+70)%70=(11-20+70)%70=61。font按front的笔误理解。

2．下三角矩阵压缩位置（3分）

按0基下标、0≤j≤i<n，A[i][j]之前共有**k=i(i+1)/2+j**个压缩元素，所以相对A[0][0]的下标偏移为k；每元素占L个单元时地址差为kL。

若原题的“之间”专指不含两个端点的元素数，则在k>0时是**k-1**，同一元素时为0。题面缺少所问量的完整文字，不能混淆偏移和严格居间元素数。下三角（含对角线）需n(n+1)/2个元素单元，原文n(n+1)少了“/2”。

3．循环队列最大容量的缺失条件（3分）

按“有m个存储单元，预留一个空槽”这一通常题意，答案为m-1。F、R相等表示空，相邻表示满。

但原文写的是“中有m个元素”，若m只表示当前已有元素数，而不是物理空间大小，就不能据此确定最大容量；须补充数组长度及判满约定。

4．朴素串匹配最坏比较数（3分）

m(n-m+1)，假定n≥m且从主串每个可能起点依次尝试完整模式。最坏每个起点都比较m次后才失配，共n-m+1个起点；数量级O(nm)。

5．广义表取原子e（3分）

原题A=(((a,b),(c,d,e))的括号不配对，需先明确广义表层次。

若A=((a,b),(c,d,e))：先令B=Head(Tail(A))=(c,d,e)，再取**Head(Tail(Tail(B)))=e**。

若A=(((a,b),(c,d,e)))（外面另有单元素表）：先令A1=Head(A)，再按上一种方法从A1取e。两种情况的Head层数不同，不能省略外层条件。

6．树的叶结点数（3分）

14。由边数n-1=Σd·nd，得n0=1+n2+2n3+3n4=1+3+2×2+3×2=14，度为1的结点数不影响该式。

7．k叉树的最大与最小深度（3分）

根位于第1层、每个结点至多k个孩子。**最大深度n；k≥2时最小深度ceil(log_k(n(k-1)+1))**。

因为深度h最多容纳1+k+…+k^(h-1)=(k^h-1)/(k-1)个结点。k=1时只能形成单链，最大、最小深度都为n。

8．Huffman树结点数（3分）

2m-1。m个叶结点、m-1个二度内部结点。

9．拓扑序列（3分）

1→4→2→3。全部给定弧都由前指向后；这里还存在链1→4→2→3，所以该序列实际上唯一。

10．最优分块查找（3分）

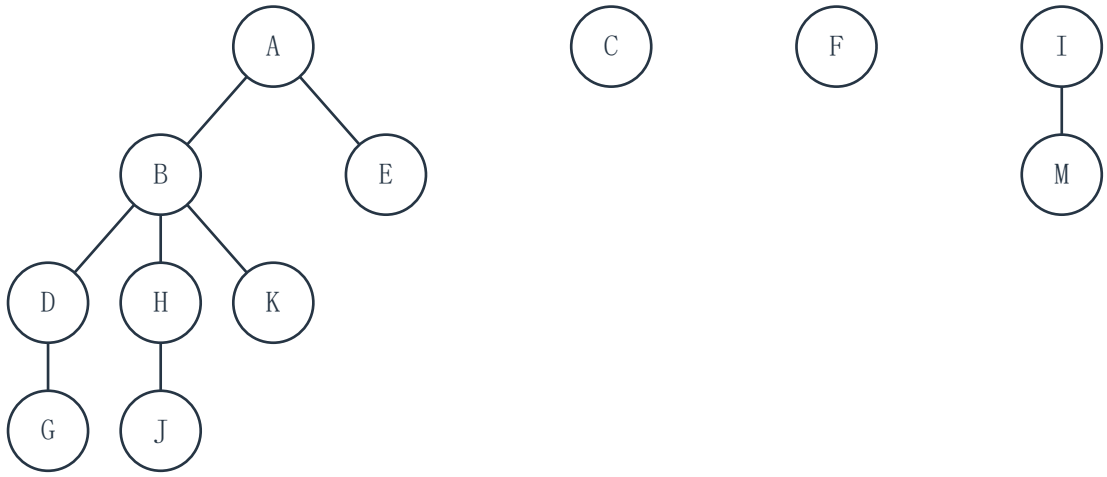
100块；每块100个记录；成功ASL=101。索引表和块内均顺序查找，等概率下ASL=(100+1)/2+(100+1)/2=101；一般使块数与块长接近sqrt(n)最优。

三、综合应用题（70分）

1．二叉树还原为森林（10分）

根的右链A→C→F→I对应四棵树的根。逐结点按“左孩子、右兄弟”还原，得到：

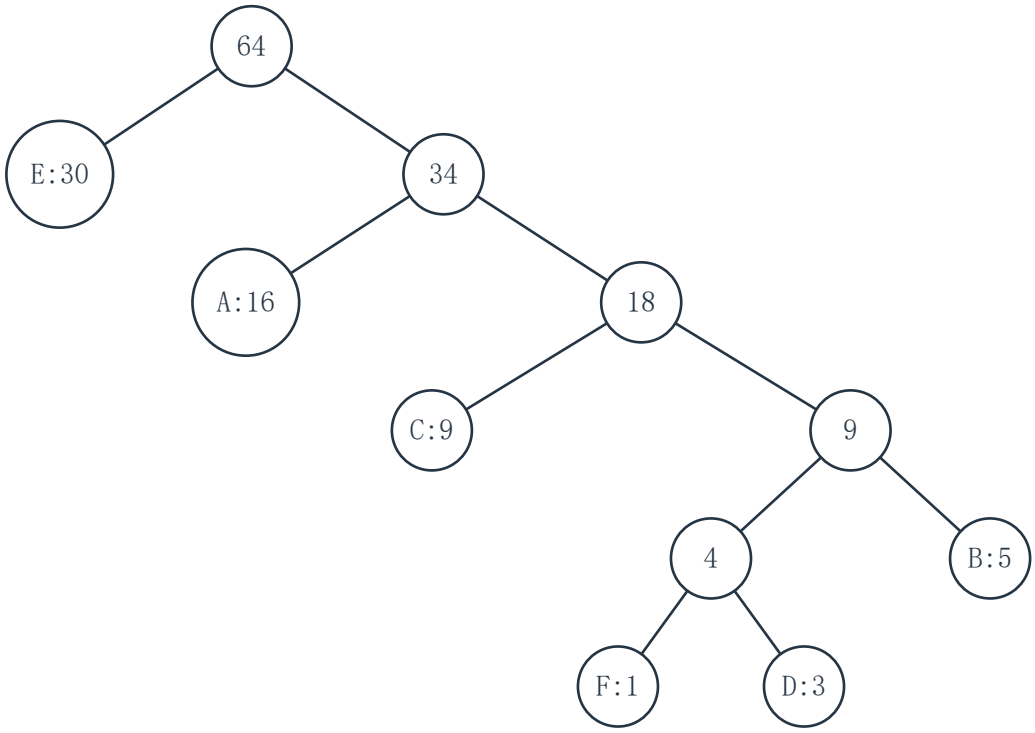
第一棵：A的孩子为B、E；B的孩子为D、H、K；D的孩子为G；H的孩子为J。第二、三棵分别是单结点C、F。第四棵的根为I，孩子为M。



森林共有四棵树，根按原二叉树的右链依次为A、C、F、I。

2．Huffman树与WPL（10分）

合并过程：F1+D3=4；4+B5=9；C9+9=18；A16+18=34；E30+34=64。同权的两个9可以交换左右子树，不影响WPL。



一棵Huffman树；同权9时把原字符C放左侧，左0右1。

为便于核对，给出按图左0右1的码字（本题未要求编码）：

字符	A	B	C	D	E	F
权值	16	5	9	3	30	1
码字	10	1111	110	11101	0	11100

WPL=16×2+5×4+9×3+3×5+30×1+1×5=129，也等于各次合并新权4+9+18+34+64=129。

3．遍历序列不一致：原题无解（10分）

不存在同时符合原题三组序列的二叉树。这不是“星号填哪一个字母”的选择问题，而是已给定条件互相矛盾：

（1）前序BC**G*只有6个位置，中序CBEAGH也只有6个位置，但后序*EDB**FA有8个位置；同一棵树的三种遍历长度必须相同。

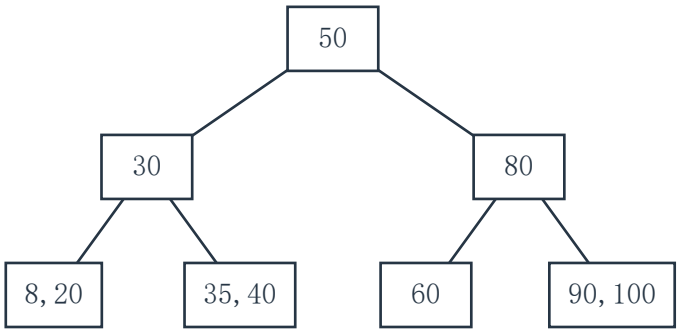
（2）前序首字母B要求根为B，后序末字母A要求根为A，二者直接冲突。

（3）后序已有D、F，而完整列出的中序CBEAGH中没有D、F。

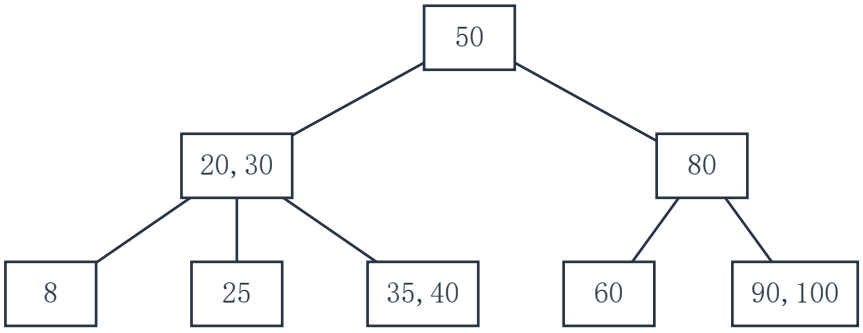
因此不能补全、重建或画出一张声称与三组原条件一致的中序线索图。须由出题方提供更正的等长序列，再按“中序分左右、先序首或后序末定根”重建。重建后，带头结点的中序线索链表还应设置头结点左指针指根，头结点右指针指中序末结点，首结点的空左链及末结点的空右链回指头结点。这里不擅自替换为其他历年题的序列。

4．3阶B树的连续操作（10分）

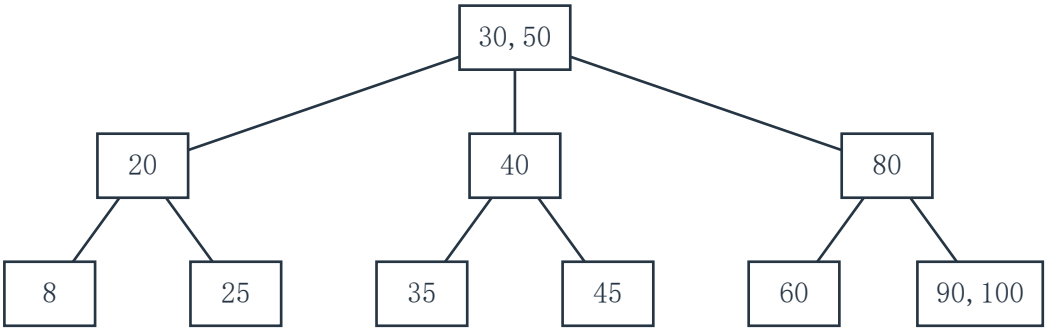
3阶B树每个结点最多2个关键字，非根结点至少1个关键字。以下四步连续执行。



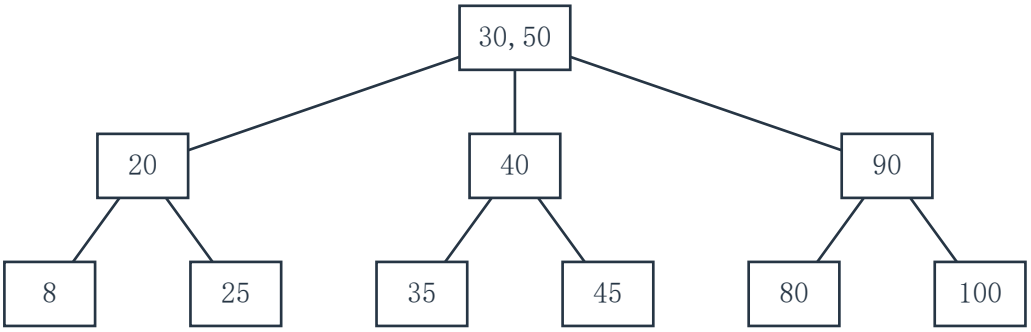
①插入90：叶[100]变为[90,100]。



②插入25：分裂[8,20,25]，20上升。



③插入45：先提升40，内部结点再分裂，30上升到根。



④删除60：向右兄弟借键，80下移、90上升。

最后一步删除叶结点中的60产生下溢；右兄弟[90,100]可以借出关键字，经父结点旋转后，80下移补空叶，90替代父键80，右兄弟保留100。最终根为[30,50]，三个内部孩子依次为[20]、[40]、[90]。

5．三种排序的全过程（10分）

（1）二路归并：从长度1的归并段开始，下表每一行是一整趟结束后的全序列。

趟次	序列
1	18 29 25 47 12 58 10 51
2	18 25 29 47 10 12 51 58
3	10 12 18 25 29 47 51 58

（2）快速排序：统一采用当前子区间首元素作枢轴、右侧先扫描的挖坑划分，先递归左区间。下表记录每一次划分，包括枢轴归位但全表次序未变化的划分。

划分	子区间（1基）	枢轴→归位下标	全序列
1	1～8	29→5	10 18 25 12 29 58 51 47
2	1～4	10→1	10 18 25 12 29 58 51 47
3	2～4	18→3	10 12 18 25 29 58 51 47
4	6～8	58→8	10 12 18 25 29 47 51 58
5	6～7	47→6	10 12 18 25 29 47 51 58

（3）大根堆排序：先由后往前下滤建堆；竖线左边为当前堆，右边为已确定的升序后缀。

状态	堆区 / 已排序区
建堆	58 47 51 29 18 12 25 10 /
取出第1个最大值	51 47 25 29 18 12 10 / 58
取出第2个最大值	47 29 25 10 18 12 / 51 58
取出第3个最大值	29 18 25 10 12 / 47 51 58
取出第4个最大值	25 18 12 10 / 29 47 51 58
取出第5个最大值	18 10 12 / 25 29 47 51 58
取出第6个最大值	12 10 / 18 25 29 47 51 58
取出第7个最大值	10 / 12 18 25 29 47 51 58

采用其他明确的快排枢轴、划分或归并调度规则，可产生不同中间状态；各步须与声明的方法一致，最终均为10、12、18、25、29、47、51、58。

6．线性探测与链地址法（10分）

关键字按32、13、49、24、38、21、4、12解释，原文“4.12”中的点按分隔号处理。H(K)=3K mod11，地址0～10。

（1）线性探测：

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	空	4	空	12	49	38	13	24	32	21	空

关键字	探测地址（依次）	探测次数
32	8	1
13	6	1
49	4	1
24	6→7	2
38	4→5	2
21	8→9	2
4	1	1
12	3	1

ASL成功=11/8。失败时从地址0～10开始的探测次数依次为1、2、1、8、7、6、5、4、3、2、1（含最后空槽），故**ASL失败=40/11**。

（2）链地址法：按输入顺序尾插。

地址	链
0	空
1	4
2	空
3	12
4	49→38
5	空
6	13→24
7	空
8	32→21
9	空
10	空

三个长度为2的链各贡献1+2次成功比较，另有两个单结点链，故**ASL成功=(3×3+2)/8=11/8**。失败需扫描对应整条链，空链比较0次，故**ASL失败=8/11**。只计关键字比较。

7．带重复标记的逐趟排序（10分）

16与16*的数值相同，星号只用于辨认记录。以下始终保留该标记。

（1）冒泡排序：从左向右比较相邻项，每趟把最大值移到未排序区末端；只在严格大于时交换。

趟次	全序列
1	2 12 16 28 10 16* 20 6 18 30
2	2 12 16 10 16* 20 6 18 28 30
3	2 12 10 16 16* 6 18 20 28 30
4	2 10 12 16 6 16* 18 20 28 30
5	2 10 12 6 16 16* 18 20 28 30
6	2 10 6 12 16 16* 18 20 28 30
7	2 6 10 12 16 16* 18 20 28 30
8	2 6 10 12 16 16* 18 20 28 30

第8趟无交换，可提前终止；若固定执行9趟，第9趟仍是同一有序序列。原16始终在16*之前，符合稳定性。

（2）快速排序：采用与上题相同的首元素枢轴、右先扫描、先左递归的挖坑划分。

划分	子区间	枢轴→归位下标	全序列
1	1～10	12→4	6 2 10 12 28 30 16* 20 16 18
2	1～3	6→2	2 6 10 12 28 30 16* 20 16 18
3	5～10	28→9	2 6 10 12 18 16 16* 20 28 30
4	5～8	18→7	2 6 10 12 16* 16 18 20 28 30
5	5～6	16*→5	2 6 10 12 16* 16 18 20 28 30

最终16*在原16之前，说明该快排实现不稳定。

（3）二路归并排序：等值时先取左段记录以保持稳定；每一行记录一整趟。

趟次	全序列
1	2 12 16 30 10 28 16* 20 6 18
2	2 12 16 30 10 16* 20 28 6 18
3	2 10 12 16 16* 20 28 30 6 18
4	2 6 10 12 16 16* 18 20 28 30

最终原16仍在16*之前。

四、算法设计与分析题（30分）

1．最近公共祖先（15分）

基本思想：后序递归。当前结点为空、等于p或等于q时直接返回；否则分别在左右子树查找。两侧都返回非空时，p、q分居两侧，当前结点即最近公共祖先；只有一侧非空则向上传递该侧结果。

题设保证p、q指向树中结点；采用结点自身也是自身祖先的通常定义，因此p==q时返回p。以下用指针的指针作为C语言输出参数r。

```
typedef struct Node {
    struct Node *LLINK;
    int INFO;
    struct Node *RLINK;
} Node;

static Node *nearest(Node *root, Node *p, Node *q) {
    if (root == 0 || root == p || root == q) return root;
    Node *left = nearest(root->LLINK, p, q);
    Node *right = nearest(root->RLINK, p, q);
    if (left != 0 && right != 0) return root;
    return left != 0 ? left : right;
}

void ANCESTOR(Node *ROOT, Node *p, Node *q, Node **r) {
    *r = nearest(ROOT, p, q);
}
```

复杂度：至多访问每个结点一次，时间O(n)；递归栈O(h)，最坏O(n)。本算法按结点指针比较，不要求INFO互不相同。评分建议：递归返回含义及分居两侧的思想4分，代码9分，复杂度2分。

2．用DFS实现DAG拓扑排序（15分）

基本思想：对尚未访问的顶点进行DFS；一个顶点的全部出邻接点处理完后，将该顶点加入完成序列。最后逆序输出完成序列，即得到拓扑序。

对于DAG中的每条弧u→v，DFS完成时间必有finish(u)>finish(v)，所以逆完成次序保证u在v之前。图可能不连通，须从每个尚未访问顶点启动DFS。下面额外用三色标记识别回边，若输入意外含环则返回0；已输出之前不会留下部分拓扑结果。

```
#include <stdio.h>
typedef struct Arc {
    int to;
    struct Arc *next;
} Arc;
typedef struct {
    int n;
    Arc *first[MAXV];
} Graph;

static int visit(const Graph *G, int u, int color[],
                 int order[], int *count) {
    color[u] = 1;                  /* 递归路径中 */
    for (const Arc *p = G->first[u]; p; p = p->next) {
        int v = p->to;
        if (color[v] == 1) return 0;
        if (color[v] == 0 && !visit(G, v, color, order, count))
            return 0;
    }
    color[u] = 2;                  /* 已完成 */
    order[(*count)++] = u;
    return 1;
}

int TopoDFS(const Graph *G) {
    int color[MAXV] = {0}, order[MAXV], count = 0;
    for (int u = 0; u < G->n; ++u)
        if (color[u] == 0 && !visit(G, u, color, order, &count))
            return 0;
    for (int i = count - 1; i >= 0; --i)
        printf("%d%s", order[i], i == 0 ? "\n" : " ");
    return 1;
}
```

复杂度：邻接表中每个顶点进入DFS一次、每条弧扫描一次，时间O(V+E)；标记、完成序列及递归栈共O(V)辅助空间。MAXV为顶点上界，实际顶点编号0～G->n-1。评分建议：逆完成序列及理由5分，遍历所有分量与记录/逆序输出8分，复杂度2分。